# Content or Context? Measuring Latent Communication Between Language Models, with Bridges Fitted on LLM-Written Text

Zhaohe Dong
University of Cambridge
dongzhaohe321418@outlook.com

November 2026

**Abstract**

Several systems connect language models through their hidden states and report that the latent channel helps. They rarely separate two questions: how much of that help is content the receiver lacks, and whether building the channel requires real text from the models' training distribution. We study both questions with an information-asymmetric protocol. A sender reads a prefix that the receiver never sees, a learned bridge passes the sender's hidden states to the receiver, and donor messages from other windows split the receiver's gain into generic, article-level and window-specific parts. Which donor is used matters a great deal. Swapping messages within a batch, where most donors come from the same document, attributes 54% of a bridge's benefit to generic context; with donors from other documents, the generic part is 6%. Between independently pretrained Pythia models, a linear bridge fitted on real text gives the receiver about the same content gain as its own prefix states injected the same way (ratio 0.99 [0.94, 1.03]). A bridge fitted only on encyclopedia-style text written by an LLM with no corpus access, with layers also selected on such text, reaches 82% [79, 85] of that reference's content gain. Bridges fitted on the models' own samples reach 44–59%, and a weight-only least-squares map 2%. In the main WikiText experiment, a message from a different document leaves the receiver worse off on average than no message when the bridge was fitted with the tested synthetic-text recipes, but not when it was fitted on real text. Real-text bridges also come close to the receiver's own states for a second Pythia pair and for GPT-2 into Pythia. They fall short when a smaller sender feeds a much stronger receiver (Pythia 160M into 410M, GPT-2 into Qwen2.5), and reach only 28% for OPT into GPT-2, although the two share a tokenizer. With a stronger sender, the LLM-text bridge's content contrast rises while no improvement in the receiver's predictions is detected, so the contrast alone can mislead. Text from a second LLM also works. At an equal number of characters, LLM-written text gives 92–95% of real text's content but about 75% of its net gain. Real- and LLM-text bridges fitted on encyclopedic text keep 73–81% of each domain's own reference content on long movie reviews, but about 30% on source code, where their net effects diverge.

Code, results and fitted bridges:
https://github.com/dongzhaohe321418-lab/content-or-context

## 1 Introduction

Several recent systems let language models communicate through hidden states instead of text. They fuse a sender's key–value cache into a receiver's (Fu et al., 2026), or share last-layer states and caches among agents built on one backbone (Zou et al., 2026). Other systems use a trained adapter to pass a sender's last-layer states to a receiver (Du et al., 2026), or affine maps to transfer features between residual streams (Chen et al., 2025).

Two questions are often left implicit. The first is how much of the benefit is content. A receiver handed a plausible representation of some preceding text may improve simply because its input now looks like a document in progress, whatever that document is. The second is what building the channel requires. The projectors in these works are trained on instruction, task or pretraining-style data, or, in LatentMAS, restricted to agents that share a backbone. Models whose data cannot be shared would need a bridge built from something else.

We study causal language models pretrained in separate runs: Pythia 70M, 160M and 410M, which share a tokenizer and training corpus (Biderman et al., 2023), and, to test generality, pairs from different families with different tokenizers (GPT-2, OPT and Qwen2.5). The sender $A$ reads a 128-token prefix. The receiver $B$ reads only the next 32 tokens, and $A$'s prefix states reach it through a bridge that injects them into its residual stream (§3.2). We measure how much this lowers $B$'s cross-entropy on the suffix and decompose the gain with *donor messages*, the same message source applied to another prefix. A prefix from another article gives the generic part; one from another window of the same article separates article-level from window-specific content. We compare every message with the *receiver-state reference*, $B$'s own prefix states injected the same way, and with $B$ reading the prefix as text.

Bridges are fitted to match $B$'s own predictive distribution, which needs probe text but no labels. The conditions differ in where the probe text comes from and, unavoidably, in how much of it there is. It is real in-domain text, encyclopedia-style articles written by an LLM with no tools or corpus access, or unconditional samples from the two models; a least-squares map between the models' embedding matrices uses none. Recent evidence that frontier models can reason directly about network weights motivates the LLM condition (Takehi et al., 2026). In small testbeds, an LLM could design a complete bridge from weights alone (Appendix F). We ask how far an LLM's world knowledge can substitute for data at the scale of pretrained models.

**Contributions.**

- **Donor controls separate content from context, if the donors come from other documents.** Like concurrent audits of latent multi-agent systems (Zhang and Emu, 2026; Cheng et al., 2026), we use messages from other examples as a control. In the main pair, donor messages from other documents show that almost all of a latent message's benefit is content. For text, the receiver-state reference and real-text bridges, the generic part is 2–7% of the benefit. Swapping messages within sorted batches instead draws donors from the same document, which removes about half of the content and misattributes it to generic context (§4.2).

- **Bridges fitted on synthetic text work, with a measurable cost.** A bridge fitted on LLM-written text and selected without corpus text reaches 82% of the receiver-state reference's content gain. Self-samples reach 44–59% and weight algebra 2%. In the main WikiText experiment, the tested synthetic-text bridges have a negative average generic part: unlike with the real-text bridge, a message about a different document leaves the receiver worse off on average than no message (§4.3). Beyond the main pair, LLM-text bridges keep 69–72% of the reference's net gain in four of the six reported model pairs. Cross-family bridging ranges from as good as within Pythia to poor, and the content contrast alone can mislead (§4.5).

- **Where and how latent messages fall short.** Injecting deeper sharply reduces what even the receiver's own prefix states deliver, and at the shallowest depth they still deliver less content than text. For every tested number of transmitted positions, the bridged message is worth less to this receiver than the same number of text tokens (§4.1, §4.4). Hand-over stitching replaces the receiver's lower computation, so it measures something else and should not be read as a test of communication (§4.6).

## 2 Related work

**Model stitching and representational similarity.** Stitching joins the lower layers of one network to the upper layers of another through a trainable layer, and has long been used to ask whether two networks compute compatible representations (Lenc and Vedaldi, 2015; Bansal et al., 2021). Chen et al. (2025) showed that affine maps between the residual streams of language models in the same family (Pythia, GPT-2, Gemma-2), fitted on OpenWebText, transfer features, probes and steering vectors. The platonic representation hypothesis (Huh et al., 2024) and unsupervised embedding translation (Jha et al., 2026) suggest that related maps exist more generally, but neither establishes the residual-stream maps needed here. We ask whether a stitch fitted without reading any corpus text improves a receiver's predictions using content it otherwise lacks.

**Weight-only alignment.** Relative representations (Moschella et al., 2023) compare independently trained latent spaces through similarities to shared anchor inputs, so they still require inputs. Permutation matching (Ainsworth et al., 2023) aligns networks from their weights alone but requires identical architectures. LatentMAS (Zou et al., 2026) maps a model's output hidden states back to its own input-embedding space with a least-squares map between the output and input embedding matrices, a training-free alignment derived from one model's weights. Our weight-only baseline applies the same idea across two models, using their stacked embedding and unembedding matrices.

**Latent communication between language models.** Several systems let language models exchange hidden states or key–value caches instead of text. Cache-to-Cache fuses one model's KV cache into another's through a learned fuser, trained on general instruction data in its main experiments (Fu et al., 2026). LatentMAS shares last-layer states and KV caches among agents built on the same backbone, without training (Zou et al., 2026). Interlat uses a trained adapter to insert a sender's last-layer states into the receiver's input-embedding sequence, including across model families, and its training objective already contrasts matched messages with messages from a different task in the same batch. It also notes that maximising this separation can harm task utility, and adds a regulariser that aligns the receiver's predictions with those under a language-space plan (Du et al., 2026). We use the matched-versus-swapped contrast for *evaluation* rather than training, and the same concern applies there: the contrast must be read next to the net gain over no message (§4.5). We also show that the choice of donor matters. A swapped message from the same article still carries much of the target's content, so a valid control must draw donors from other documents (§3.2). Two concurrent audits use similar controls on latent multi-agent pipelines and reasoning benchmarks. Zhang and Emu (2026) separate the effect that a message generated from a different example retains from the gain attributable to example-specific content. Cheng et al. (2026) compare relayed KV caches with the cache of a different example from the same batch, and with zeroed and random caches, and find that a large cache effect need not depend on pairing. Their donors are independent benchmark items. Our setting adds contiguous text, where a donor from the same document shares content with the target, and bridges between independently trained models.

**Data-free transfer.** Data-free knowledge distillation synthesises inputs from a trained teacher, for instance by inverting it under batch-norm statistics (Yin et al., 2020). Our self-sampling baseline is the language-model analogue: the two models generate their own probe text. The LLM-written condition replaces model inversion with an external model's world knowledge. It is motivated by recent evidence that frontier models can read and edit network weights directly (Takehi et al., 2026).

# 3 Setting and method

## 3.1 Latent bridges

Let $A$ and $B$ be independently trained causal language models with a shared tokenizer. Write $h^A_\ell(x) \in \mathbb{R}^{n\times d_A}$ for the residual stream of $A$ after layer $\ell$ on an $n$-token input $x$, and similarly for $B$. A *bridge* $f : \mathbb{R}^{d_A} \to \mathbb{R}^{d_B}$ is applied position-wise. It is affine, $f(h) = hM + c$, unless stated otherwise; for the capacity study (§4.6) we add a residual MLP branch, $f(h) = hM + c + \mathrm{GELU}\big(\mathrm{LN}(h)W_1 + b_1\big)W_2 + b_2$, with hidden width 2048 and $W_2$ and $b_2$ initialised to zero. $B$ resumes computation at layer $L_B + 1$ from the bridged states. All model weights are frozen; only $f$ is trained.

## 3.2 Two settings

**Hand-over ("full").** $B$ receives every position from $A$: $h^B_{L_B} \leftarrow f(h^A_{L_A}(x))$, and we score $B$'s next-token predictions on $x$. This classical stitching test measures how much of a single model's computation survives being split across two networks. It cannot establish that anything was *communicated*, because $B$ could have read $x$ itself.

**Information-asymmetric communication ("comm").** A window $s$ of $P + S + 1$ tokens is split into a prefix $s_{1:P}$ and a suffix. $A$ reads only the prefix. $B$ runs layers $1..L_B$ on the suffix $s_{P+1:P+S}$ alone, producing states $u \in \mathbb{R}^{S\times d_B}$. Position indices are $P+1..P+S$, so the rotary phases match those of the full window. Layers $L_B + 1..$ then run on the concatenation $[\, f(h^A_{L_A}(s_{1:P}))\,;\, u\,]$. We score $B$'s predictions of $s_{P+2:P+S+1}$. Because $B$'s lower layers never see the prefix, the message is the only route by which prefix information can reach $B$. We use $P = 128$ and $S = 32$ throughout.

**References and controls.** On identical windows we report:

- **No context**: $B$ on the suffix alone.
- **Text**: $B$ reading the whole window, i.e., a channel that forwards the prefix verbatim.
- **Receiver-state reference**: $B$'s own layer-$L_B$ states for the prefix, computed by $B$ reading the prefix alone and injected exactly as a bridged message would be. It represents what $B$ itself would send and has not been proved to be an upper bound.
- **Donor messages**: window $i$ receives a message from the same source, computed from another window's prefix. The donor comes from a *different article* or *another window of the same article*. Articles are delimited by WikiText's top-level headings, and we use five independent random draws of each kind per window.

These give a three-way decomposition of the reduction in $B$'s suffix cross-entropy relative to no context: a *generic* part (no context minus other-article donor), an *article-level* part (other-article minus same-article donor) and a *window-specific* part (same-article donor minus matched message). We call the sum of the last two the *content* gain. As differences in expected log-score, they measure the message's usefulness to this fixed receiver $B$, not mutual information.

## 3.3 Fitting bridges without real corpus text

Every fitted bridge minimises the per-token KL divergence from $B$'s own predictive distribution to $B$'s distribution through the bridge,

$$\mathcal{L}(f) = \mathbb{E}_{s\sim\mathcal{D}_{\text{probe}}} \frac{1}{|T|} \sum_{t\in T} \mathrm{KL}\big(p_B(\cdot \mid s_{\le t}) \,\big\|\, p_{B\circ f\circ A}(\cdot \mid s_{\le t})\big), \tag{1}$$

where $T$ ranges over all positions (*hand-over objective*) or over the suffix positions of the comm setting, with $B$'s full-context distribution as the target (*comm objective*). The objective needs no labels, only probe text on which both models can be run. What differs between conditions is $\mathcal{D}_{\text{probe}}$:

- **Real text** (upper reference): windows from WikiText-103 training articles (Merity et al., 2016), the domain of the test set.
- **LLM-written text** (synthetic, no real corpus): encyclopedia-style articles written by an LLM (Claude Opus 5.5) with no tools or access to any corpus. The prompt supplied only a style instruction and a topic area, and told the LLM to invent the content. We collected 64 articles (1.03M characters), one per topic area. The main experiments use 24 of them (382k characters, 88k Pythia tokens). Section 4.5 varies the amount.
- **Self-samples** (synthetic, from the two models themselves): unconditional samples from $A$ and $B$ themselves at temperature 1.
- **Weight-only anchors** (no fitting): $M$ solves $\min_M \|[E_A; U_A]M - [E_B; U_B]\|_F$ over the token embedding and unembedding matrices, $c = 0$. This map also initialises every fitted bridge.

LLM-written text is still data: it is synthetic text generated by a teacher model and carries that model's knowledge of language and of the world. What it avoids is real corpus text. Fitting an LLM-written or self-sampled bridge reads no corpus text, and the LLM that wrote the main corpus ran with all tools disabled and no access to files or the network (the second LLM's variant corpus was generated in a read-only sandbox; §6). This does not rule out WikiText-like or even WikiText text in the LLM's pretraining; we checked only verbatim overlap with the test and validation splits (Appendix E). Layer selection in the main experiments used real validation text, and §4.1 repeats it with LLM-written text only.

### 3.4 Protocol details

We use final-step Pythia checkpoints (Biderman et al., 2023): 70M ($d = 512$, 6 layers), 160M ($d = 768$, 12 layers) and 410M ($d = 1024$, 24 layers). They share a tokenizer and training data but come from separate runs. Bridges are trained with Adam (learning rate $10^{-3}$, batches of 16 windows, 600 steps) on 4096 probe windows of 161 tokens drawn with replacement from the probe text. Self-sampled bridges use 512 generated sequences of the same length, whose 82k tokens are comparable to the 88k tokens of the 382k-character LLM-written text.

Test sets contain 512 non-overlapping windows from the first 6000 lines of the WikiText-103 test split, covering 58 articles. Layer pairs are chosen on the validation split, never on test windows. Because windows from one article are not independent, we use a hierarchical bootstrap (4000 draws) for intervals unless stated otherwise. It resamples whole articles and, within each draw, fitting seeds and donor draws; per-seed estimates are reported alongside. Paired comparisons and per-position or truncation analyses that resample only articles are labelled as conditional on the saved fits and donors. Main-experiment fits use three seeds, which change the probe windows (or samples) and the batch order; diagnostics and layer sweeps use one seed. All runs used one Apple M5 Pro GPU with the Metal backend in float32.

## 4 Results

### 4.1 Choosing where to bridge

Everything $B$ computes on the suffix below the injection depth $L_B$ is computed without the prefix. Injecting $B$'s own prefix states shows what this costs (Appendix A). With pythia-160m as $B$, this reference reaches 34.3% next-token accuracy at $L_B = 1$ but only 26.2% at $L_B = 6$, against 37.1% with the full prefix and 24.0% with none. Mid-depth hand-over, the usual stitching configuration, therefore leaves little room for communication.

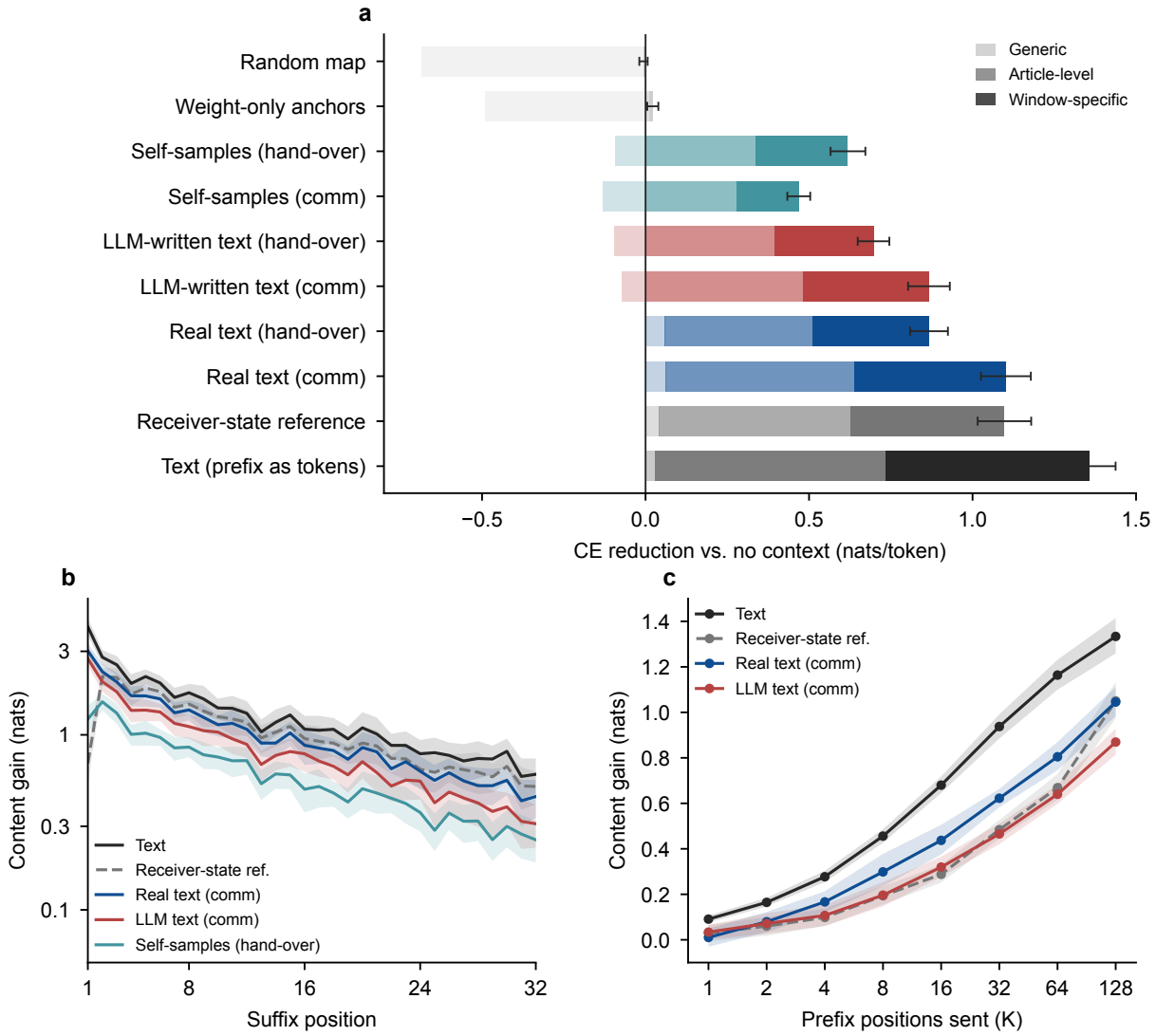



Figure 1: **What a latent message is worth to the receiver** (pythia-70m layer 2 $\rightarrow$ pythia-160m layer 1; $B$ sees only the 32-token suffix). **a**, Reduction in $B$'s suffix cross-entropy relative to no context on 493 test windows from 54 WikiText-103 articles (windows inside one article that have a same-article donor), decomposed with donor messages (§3.2): generic (a message from another article), article-level (another window of the same article, minus another article) and window-specific (own window, minus a same-article window). Negative generic parts are drawn left of zero. Error bars: 95% interval of the content part (article-level + window-specific) from a bootstrap over articles, fitting seeds and donor draws. **b**, Content gain at each suffix position, shown on a log scale. **c**, Content gain when only the last $K$ prefix positions are sent; for text, $B$ reads the last $K$ prefix tokens. Both panels use all 512 windows and one other-article donor per window, with pointwise 95% article-bootstrap bands.

Using a short real-text fit, we selected $(L_A, L_B)$ on the WikiText *validation* split (Appendix B). The pair $L_A = 2$, $L_B = 1$ minimised validation cross-entropy, with $L_A = 1$–3 close behind. Mapping $A$'s final hidden state, taken after its output layer norm, gave the worst validation cross-entropy at both $L_B$ (4.64 and 4.55, against 4.03). A check run after the main experiments showed that layer selection also works without

corpus text. We fitted on the 24 LLM-written articles used in the main experiments and validated on the other 40 LLM-written articles. The same pair was selected, $L_A = 2$, $L_B = 1$, with validation CE 3.531 against 3.593 for the next best pair. The LLM-text pipeline below therefore uses no real corpus text for bridge fitting or layer selection.

### 4.2 A valid control needs donors from other documents

Swapping messages between examples is a natural control for latent communication (cf. Du et al., 2026), but its interpretation depends on which examples supply the donors. Our first analysis rolled messages within each batch of 16 windows. Because test windows are sorted by text position, 83% of those donors came from the same article (427 of 512), and the control attributed 54% of the real-text bridge's benefit (0.59 of 1.10 nats) to "generic" context. With donors from other articles, the same bridge's generic part is 0.06 nats [0.03, 0.10], or 6% of its benefit (Figure 1a). For text, it is 0.03 nats [−0.00, 0.06] of a 1.36-nat benefit. Nearly all of the benefit of a real prefix or a real-text bridge is therefore content, and about half of that content is shared across windows of the same article (for text, 0.71 nats article-level and 0.62 window-specific). A same-article donor removes the shared half along with the generic part.

The decompositions below draw five donors per window from other articles and five from the same article, using only windows that lie inside one article. They are evaluated on the 493 recipient windows that lie inside one article and have a same-article donor, so that the generic, article-level and window-specific parts sum exactly to the total. Intervals come from a bootstrap that resamples articles, fitting seeds and donor draws.

### 4.3 Bridges fitted on synthetic text deliver most of the receiver-state reference

Table 1 compares every message source with the *receiver-state reference*, $B$'s own prefix states injected the same way. The reference is what $B$ itself would have sent. It is not an upper bound, because a bridge could supply states better suited to the spliced computation. We report two ratios to it: the ratio of content gains isolates what depends on the particular prefix, and the ratio of total gains also counts how the receiver fares with an irrelevant message.

*A real-text bridge comes close to the reference.* With the communication objective, its content gain is 1.04 nats against the reference's 1.06, a ratio of 0.99 [0.94, 1.03] (per seed 0.94, 1.01 and 1.01), and its total gain is 1.00 [0.97, 1.04] of the reference's. A linear map from a 70M model's layer-2 states thus lets a 160M receiver predict the suffix about as well as its own layer-1 states would. The profiles over suffix positions differ (panel b): the bridged message is far more useful at the first position and slightly less useful later on. Perhaps $A$'s layer-2 states have mixed in more of the prefix than $B$'s layer-1 states, but we did not test this.

*LLM-written text gets 82% of the reference's content.* The communication-objective bridge uses 382k characters of encyclopedia-style text written by an LLM with no access to any corpus. Its content gain is 0.86 nats, 0.82 [0.79, 0.85] of the reference (per seed 0.80, 0.81 and 0.85) and 0.83 of the real-text bridge. The total-gain ratio is lower, 0.72 [0.68, 0.76], because synthetic-text bridges have a *negative* generic part. Given a message about a different article, $B$ does worse on average than with no message at all ($-0.07$ nats [$-0.11$, $-0.04$] for LLM-written text, $-0.13$ for self-samples), whereas real-text bridges still help slightly.

The harm is not a general weakness of the receiver or the hidden-state interface in this pair and domain. An unrelated article's prefix given *as text* does not worsen $B$'s predictions on average; its generic part is $+0.03$. The same holds for an unrelated article's receiver-state reference ($+0.04$) and an unrelated message through the real-text bridge ($+0.06$). Paired per window with the text channel, the irrelevant-message effect differs by $+0.01$ [$-0.00$, 0.03] for the reference and $+0.03$ [0.02, 0.05] for the real-text bridge, but by $-0.10$ [$-0.12$, $-0.08$] for the LLM-text bridge and $-0.16$ [$-0.17$, $-0.14$] for the self-sample bridge. These intervals come from an article bootstrap, conditional on the saved fits and donor draws. Averaged over seeds and donors, an irrelevant message worsens CE in 66% of windows for the LLM-text bridge and 81% for the self-sample bridge, against 48% for irrelevant text.

Table 1: Communication setting, pythia-70m L2 → pythia-160m L1, on 493 test windows (54 articles) that lie inside one article and have a same-article donor. Suffix CE in nats/token; gains relative to $B$ with no context (CE 4.930). Generic + content = total gain; the split of content into article-level and window-specific parts is shown in Figure 1a. Content and net ratios divide a bridge's content and total gains by those of the receiver-state reference. 95% intervals use a bootstrap over articles, fitting seeds and donor draws, with 4000 draws. Fitted bridges average three seeds. All conditions average five donor draws of each kind.

| Message source (objective) | Total | Generic | Content | Content ratio | Net ratio |
|---|---|---|---|---|---|
| Random map | −0.690 | −0.684 | −0.006 [−0.019, 0.007] | −0.01 [−0.02, 0.01] | −0.63 [−0.68, −0.59] |
| Weight-only anchors | −0.469 | −0.491 | 0.022 [0.005, 0.039] | 0.02 [0.00, 0.04] | −0.43 [−0.46, −0.40] |
| Self-samples (hand-over) | 0.525 | −0.093 | 0.618 [0.566, 0.672] | 0.59 [0.55, 0.62] | 0.48 [0.45, 0.52] |
| Self-samples (comm) | 0.338 | −0.129 | 0.467 [0.434, 0.504] | 0.44 [0.42, 0.47] | 0.31 [0.29, 0.33] |
| LLM-written text (hand-over) | 0.601 | −0.096 | 0.697 [0.649, 0.746] | 0.66 [0.64, 0.68] | 0.55 [0.52, 0.57] |
| LLM-written text (comm) | 0.792 | −0.073 | 0.865 [0.803, 0.930] | 0.82 [0.79, 0.85] | 0.72 [0.68, 0.76] |
| Real text (hand-over) | 0.866 | 0.059 | 0.807 [0.750, 0.866] | 0.76 [0.74, 0.80] | 0.79 [0.76, 0.82] |
| Real text (comm) | 1.102 | 0.062 | 1.040 [0.963, 1.116] | 0.99 [0.94, 1.03] | 1.00 [0.97, 1.04] |
| Receiver-state reference | 1.097 | 0.042 | 1.055 [0.974, 1.138] | 1 | 1 |
| Text (prefix as tokens) | 1.357 | 0.028 | 1.329 [1.247, 1.410] | 1.26 [1.23, 1.29] | 1.24 [1.21, 1.27] |

For this pair and domain, then, the extra fragility is specific to bridges fitted on synthetic text. It is not universal. In other settings irrelevant messages also hurt through real-text bridges (generic part $-0.07$ with pythia-410m as receiver, $-0.02$ on reviews and $-0.35$ on code), and even as text ($-0.18$ with pythia-410m as receiver and $-0.26$ on code; Appendix D). Of the LLM-text bridge's 0.31-nat shortfall relative to the real-text bridge, 0.17 [0.14, 0.21] is content and 0.13 [0.11, 0.16] generic gain.

*Self-samples and weight algebra fall far short.* Bridges fitted on samples from $A$ and $B$ themselves reach 0.44 [0.42, 0.47] of the reference's content with the communication objective and 0.59 [0.55, 0.62] with the hand-over objective. The best LLM-text bridge exceeds the best self-sample bridge by 0.25 nats [0.21, 0.28] of content; under matched objectives the margins are 0.40 [0.36, 0.44] for communication and 0.08 [0.04, 0.11] for hand-over.

The probe pools differ in size as well as source: 8.8M characters of real text, 382k characters (88k tokens) of LLM text and 512 sampled sequences (82k tokens) of self-samples. These comparisons are therefore between recipes with unequal budgets; §4.5 compares real and LLM-written text at equal size. The weight-only least-squares map reaches a content gain of 0.02 nats [0.01, 0.04], 2% of the reference, with a generic loss of 0.49 nats. The random map shows no detectable content gain ($-0.01$ [$-0.02$, 0.01]).

### 4.4 Per transmitted position, the latent message is worth less than text

Hidden states are often assumed to carry more than their corresponding tokens, but for this receiver and these bridges our measurements do not support that. At full length, the receiver-state reference delivers 79% of text's content gain (1.06 against 1.33 nats; Table 1). One difference from text is that, with the message, $B$'s first layer processes the suffix without the prefix.

Sending only the last $K$ prefix positions gives the same picture (Figure 1c). At every tested $K \in \{1, 2, 4, \ldots, 128\}$, $K$ text tokens deliver more content gain than $K$ bridged positions, by 0.08 nats [0.04, 0.12] at $K = 1$ up to 0.36 [0.30, 0.43] at $K = 64$ for the real-text bridge. All eight Bonferroni-adjusted intervals exclude zero (smallest lower bound 0.02); they resample articles and are conditional on the saved bridges and one other-article donor draw. The result is operational: it concerns usefulness to this receiver, not information density.

Short messages also change other things: the bridges were fitted only on 128-position messages, and truncation places a mid-document state at sequence position 0. Absolute cross-entropy with a short la-

tent message is *worse* than with no context (Appendix C). Bridges trained for short messages may behave differently, as may receivers that inject at several layers or into key–value caches (Fu et al., 2026).

Matching the number of positions does not match communication costs. A 128-position message costs 262 kB in float32 if $A$'s 512-dimensional states are sent and the bridge is applied at the receiver (131 kB in float16), or 393 kB if the bridged 768-dimensional states are sent. The same 128 token ids, packed as 16-bit integers, cost 256 bytes, so the latent message is about three orders of magnitude larger.

Even in an idealised split implementation, the receiver still runs 11 of its 12 layers on every message position, roughly the computation of reading the prefix as text: a dense-layer estimate gives $2 \times 85$M non-embedding parameters $\times 128$ positions $\approx$ 22 GFLOP either way, and the sender adds two of its layers. Our implementation runs both models in full and overwrites states, so it does not realise even this split implementation. We did not measure latency, and this design shows no communication-efficiency advantage. The case for latent channels rests on settings where the sender would otherwise *generate* text token by token, which motivates Fu et al. (2026) and Zou et al. (2026); we do not test such settings.

### 4.5 How far the findings generalise

Table 2 repeats the measurement across model pairs, model families and test domains. The content contrast can grow because matched messages help more or because irrelevant messages hurt more, so for each bridge we report both its content ratio and its *net* ratio: the total reduction in cross-entropy relative to no context, divided by the reference's. Appendix D lists the populations and protocol differences.

**The receiver's own states are matched only in some pairs.** In-domain, the real-text bridge's net ratio is close to 1 when pythia-70m, pythia-410m or GPT-2 sends to pythia-160m (1.00, 0.99 and 1.00). It is lower, 0.87 [0.82, 0.92], when pythia-160m sends to pythia-410m, a receiver whose own states are far more useful (1.51 nats of content, 93% of what text gives it). The shortfall could come from the sender, the bridge's capacity or optimisation, or the restricted layer search, since the selected sender layer was the deepest candidate; we cannot tell these apart. The main pair shows that a less capable sender is not by itself an obstacle.

**No net improvement was detected with a stronger sender.** We replaced pythia-70m by pythia-410m as sender to the same receiver, on identical windows and donors. The LLM-text bridge's content ratio rises from 0.82 to 0.89, but the net gain does not change detectably for any probe source, and its point estimates are slightly lower. For real text, LLM text and self-samples, the net gain changes by $-0.01$ [$-0.06$, 0.04], $-0.03$ [$-0.07$, 0.01] and $-0.00$ [$-0.09$, 0.11] nats, and the content gain by $+0.03$ [$-0.03$, 0.11], $+0.07$ [0.04, 0.11] and $+0.11$ [$-0.01$, 0.22]. The bootstrap pairs fitting seeds across senders, which share probe and minibatch seeds, and uses identical donor draws. Only the LLM-text bridge's content change is detectably positive, and in these fits its larger contrast reflects worse predictions with irrelevant messages rather than better ones with relevant messages. Read alone, the content contrast would suggest an improvement that the net gain does not show.

**Other model families.** For pairs with different tokenizers, windows are defined in the receiver's tokenisation and the sender reads the same prefix text in its own (Appendix D). We use only the communication objective, which needs no token alignment. From GPT-2 (layer 4) into pythia-160m (layer 1), on the main pair's test windows, the real-text bridge again comes close to the receiver's own states (content ratio 1.02 [0.98, 1.05], net ratio 1.00 [0.97, 1.03]). LLM-written text reaches 0.87 and 0.70, self-samples 0.70 and 0.54, and the weight-only map, fitted on 36,938 shared token strings, 0.02 in content. Under our protocol, this pair bridges about as well as two Pythia models despite coming from different organisations, training corpora and tokenizers. From GPT-2 into Qwen2.5-0.5B, a much stronger receiver whose own layer-1 states already give 96% of the content of text, no bridge comes close. Content ratios are 0.50 [0.25, 0.76] for real

Table 2: Generalisation. For each bridge, content gain (other-document donor minus matched message) and net gain (no context minus matched message), each relative to the receiver-state reference of the same receiver on the same windows; reference content in nats. 95% intervals over documents, fitting seeds and donor draws. All bridges use the communication objective; layer pairs were selected on validation text for each model pair. The code and review rows evaluate the main pair's bridges, fitted on WikiText-like text, without refitting. Too few review windows have a same-document donor to split content into article-level and window-specific parts (Appendix D).

| Setting | Probe text | Ref. content | Content ratio | Net ratio |
|---|---|---|---|---|
| Pythia 70M→160M (main) | Real text | 1.06 | 0.99 [0.94, 1.03] | 1.00 [0.97, 1.04] |
| | LLM text | | 0.82 [0.79, 0.85] | 0.72 [0.68, 0.76] |
| | Self-samples | | 0.44 [0.42, 0.47] | 0.31 [0.29, 0.33] |
| Pythia 410M→160M | Real text | 1.06 | 1.02 [0.98, 1.06] | 0.99 [0.96, 1.03] |
| | LLM text | | 0.89 [0.86, 0.92] | 0.70 [0.67, 0.73] |
| | Self-samples | | 0.55 [0.45, 0.65] | 0.31 [0.22, 0.40] |
| Pythia 160M→410M | Real text | 1.51 | 0.85 [0.79, 0.91] | 0.87 [0.82, 0.92] |
| | LLM text | | 0.77 [0.75, 0.80] | 0.69 [0.67, 0.72] |
| | Self-samples | | 0.27 [0.24, 0.34] | 0.14 [0.09, 0.17] |
| GPT-2→Pythia 160M | Real text | 1.05 | 1.02 [0.98, 1.05] | 1.00 [0.97, 1.03] |
| | LLM text | | 0.87 [0.78, 0.93] | 0.70 [0.61, 0.76] |
| | Self-samples | | 0.70 [0.66, 0.74] | 0.54 [0.51, 0.58] |
| GPT-2→Qwen2.5 0.5B | Real text | 1.58 | 0.50 [0.25, 0.76] | 0.62 [0.44, 0.80] |
| | LLM text | | 0.57 [0.46, 0.67] | 0.43 [0.34, 0.52] |
| OPT 125M→GPT-2 | Real text | 1.40 | 0.28 [0.26, 0.30] | 0.65 [0.60, 0.71] |
| | LLM text | | 0.28 [0.25, 0.31] | 0.27 [0.22, 0.32] |
| Code (70M→160M) | Real text | 1.26 | 0.31 [0.26, 0.34] | 0.03 [−0.03, 0.10] |
| | LLM text | | 0.30 [0.27, 0.32] | −0.10 [−0.16, −0.03] |
| | Self-samples | | 0.30 [0.28, 0.31] | 0.21 [0.19, 0.24] |
| Reviews (70M→160M) | Real text | 0.43 | 0.81 [0.76, 0.87] | 0.71 [0.65, 0.76] |
| | LLM text | | 0.73 [0.70, 0.76] | 0.63 [0.60, 0.67] |
| | Self-samples | | 0.46 [0.43, 0.50] | 0.46 [0.42, 0.50] |

text and 0.57 [0.46, 0.67] for LLM-written text, and the seeds differ markedly (real text 0.75 and 0.26). Our protocol handicaps this pair. The sender reads only the last 90 tokens of its re-tokenised prefix ($P_A = 90$, a length set by digit-heavy prefixes that Qwen splits into single-digit tokens), on average 74% of the prefix, while the reference uses all of it. Six hundred steps may also be too few for a receiver with a 152k-token vocabulary. Two seeds cannot separate these causes, so this pair shows only that our protocol does not reach a much stronger receiver's own states. OPT-125M and GPT-2 have similar sizes and share a tokenizer, yet bridges from OPT-125M into GPT-2 perform poorly. They deliver only 0.28 [0.26, 0.30] of the receiver's own states' content with real text and 0.28 [0.25, 0.31] with LLM-written text, consistently across seeds, and the real-text bridge has a large positive generic part (about 36% of its net gain). The layer search covered only $L_A \in \{1, 2, 4\}$. Because GPT-2 uses absolute position embeddings, the generic and net parts may include a position-offset effect (Appendix D); matched and donor messages, however, are compared at identical position offsets. Cross-family bridging succeeded for one pair, partly for one and poorly for one, so it is not uniformly easy.

**Other domains.** The main pair's bridges were fitted for encyclopedia-like text. On long IMDB reviews (at least 322 tokens, 30% of the test split), the domain's own reference is weaker, with 0.43 nats of content,

and the bridges keep most of its content and net gain. Their content and net ratios are 0.81 and 0.71 for real text, 0.73 and 0.63 for LLM text, and 0.46 and 0.46 for self-samples.

On Python source code, all three retain about 30% of the reference's content, but their net effects diverge. The self-sample bridge still helps (net ratio 0.21 [0.19, 0.24]), the real-text bridge's effect is unresolved (0.03 [$-0.03$, 0.10]), and the LLM-text bridge leaves the receiver worse off than no message ($-0.10$ [$-0.16$, $-0.03$]). On code, even an unrelated file's prefix given as text hurts the receiver (generic part $-0.26$). Relative performance falls sharply on this corpus, and which bridge is useful there is not predicted by which is best in-domain.

**Text source and amount.** To see whether the LLM-written result depends on one model or one batch of text, we generated two further corpora on 24 other topic areas, each truncated to 371k characters: encyclopedia articles from a different LLM (GPT, same prompt), and fiction with narration and dialogue from the first LLM. With two fitting seeds each, content ratios are 0.77 [0.74, 0.81] for the first LLM's encyclopedia text, 0.69 [0.66, 0.72] for the second LLM's and 0.55 [0.52, 0.58] for fiction. The net ratios are 0.55, 0.52 and 0.31, and the pairwise content differences 0.08 [0.06, 0.11], 0.23 [0.19, 0.28] and 0.15 [0.12, 0.18] nats. A second LLM's encyclopedia text therefore also yields a working bridge, and both encyclopedia corpora do better than this fiction corpus.

The comparison does not separate author from register. The corpora hold 72k to 96k tokens for the same number of characters and differ in which topics survive truncation (20 to 24 per corpus). Each is a single generated sample, and we lack the GPT-fiction cell. We compared real and LLM-written text at equal numbers of characters, using the first 383k or 1.03M characters of each corpus and two seeds. Here real text comes from the start of the ordered WikiText training split, whereas the main bridge samples the full 8.8M-character pool. Real text reaches content ratios of 0.87 [0.82, 0.93] and 0.91 [0.88, 0.93] and net ratios of 0.92 and 0.96. LLM-written text reaches 0.80 [0.78, 0.83] and 0.86 [0.81, 0.90] in content but only 0.69 and 0.73 net. At equal character counts, LLM-written text therefore gives 92% and 95% of real text's content gain, but only 75% and 76% of its net gain. The corpora are fixed, not random subsets, so this experiment does not attribute the main gap to quantity.

**Bridge capacity.** Restricting the main pair's bridge to rank 16 or 64 (one seed) gives content ratios of 0.22 and 0.61 for real text and 0.34 and 0.55 for LLM-written text, against 0.94 and 0.80 for the same seed's full-rank bridges. The real-text low-rank bridges keep a much larger share of net gain (0.51 and 0.76) than of content, which means a large generic part.

Adding an MLP branch of width 2048 raises the same-seed real-text bridge's content ratio from 0.94 to 0.97 and net ratio from 0.97 to 1.00. For the LLM-text bridge, the content ratio rises from 0.80 to 0.86 and the net ratio from 0.72 to 0.78. With one seed, and with capacity, parameterisation and optimisation varied together, we read these results as descriptive. Relative to the same-seed full-rank bridge, rank 16 keeps 23% of the content for real text and 43% for LLM text, and rank 64 keeps 64% and 69%. In this comparison, extra capacity helps the LLM-text bridge somewhat more than the real-text bridge.

### 4.6 Hand-over stitching answers a different question

In the classical hand-over setting, $B$ reads the tokens, but the bridged $A$ state replaces every layer-$L_B$ state. Bridged systems score below $A$ alone on accuracy, both here and on two small testbeds (Appendix F). Table 3 varies optimisation length, bridge capacity and layer pairing for 70M→160M. It uses a real-text probe, one seed and 512 test windows. We interpret this diagnostic descriptively because it uses a single seed and checkpoints chosen post hoc.

No configuration reaches $B$ alone, and none exceeds $A$'s accuracy. The longest-trained non-linear bridge nevertheless has *lower* cross-entropy than $A$ alone, at 4.535 against 4.626. Whether a stitched system "performs worse than the sender" therefore depends on the metric and on training, and this observation does

Table 3: Hand-over setting, pythia-70m → pythia-160m: next-token accuracy (%) / cross-entropy (nats) over all 128 positions; one seed. $A$ alone: 25.65 / 4.626; $B$ alone: 30.60 / 4.074. Bridges start from the weight-only map and are trained with Adam at a constant learning rate of $10^{-3}$; the 1500- and 3000-step entries are checkpoints of the same run.

| | 600 steps | 1500 steps | 3000 steps |
|---|---|---|---|
| L3→L6, linear | 21.67 / 4.785 | 23.61 / 4.615 | 22.69 / 4.706 |
| L3→L6, linear + MLP (2048) | 22.38 / 4.710 | 23.82 / 4.582 | 24.64 / 4.535 |
| Depth-matched pairs, linear, 600 steps: | | | |
| L1→L2 / L2→L4 | 22.44 / 4.786 | 19.71 / 4.998 | |
| L4→L8 / L5→L10 | 23.00 / 4.712 | 23.40 / 4.702 | |
| L6 (final, after the output layer norm)→L12 | 13.70 / 6.178 | | |

not establish a bound: in principle, a downstream decoder can use a representation better than the sender's own upper layers do.

Training length and capacity matter too. At 3000 steps, the non-linear bridge closes 75% of the accuracy gap between the 600-step linear bridge and $A$ alone. With a constant learning rate, the linear map's accuracy is not monotone in training length: 23.61 at 1500 steps and 22.69 at 3000. Comparisons between stitches based on single checkpoints therefore require caution.

$A$'s final hidden state, taken after its output layer norm, is a poor source both here and in the communication setting (§4.1). We did not test the pre-normalisation final residual state, so these results say nothing about the information present before $A$'s last normalisation.

In the communication setting, $B$ keeps its own suffix computation and receives only what it lacks. We ask whether the message improves $B$ beyond the benefit of any message. Comparing the stitched system with either model answers a different question.

# 5 Discussion

**Evaluating latent channels.** The donor control costs one extra forward pass per donor message, yet the choice of donor changes the conclusion. In the main pair, donors from other documents show that nearly all of the benefit of text, of the receiver's own states and of real-text bridges is content. Donors from the same document silently count about half of that content as generic. Work that contrasts matched with swapped messages, whether for training (Du et al., 2026) or for evaluation, should therefore say where the swapped messages come from. We also recommend reporting the receiver's own states injected the same way. In our setting this reference shows how quickly the benefit of a message falls with injection depth, and that even at the shallowest depth it delivers less content than the corresponding text.

**Building bridges without real corpus text.** An LLM with no access to any corpus wrote encyclopedia-style probe text that was good enough for a bridge delivering 82% of the receiver-state reference's content. The models' own samples and weight algebra are much weaker substitutes. In the main WikiText experiment, however, the tested synthetic fitting recipes cost more than content. With these bridges, a message about a different document leaves the receiver worse off on average than no message at all, whereas with the real-text bridge or plain text it does not. The pattern does not hold everywhere: in other settings irrelevant messages also hurt through real-text bridges and plain text, and on reviews the self-sample bridge's generic part is not negative (§4.5). Because we compare complete recipes, we do not isolate synthetic provenance as the cause. For systems that may receive irrelevant messages, this is a robustness cost that a comparison with no context hides, and behaviour under adversarially chosen messages remains untested. Whether a small amount of in-domain text would remove the cost is the most direct open question. The generalisation exper-

iments argue for reporting net gain next to any matched-versus-swapped contrast: with a stronger sender, the LLM-text bridge's contrast rose while no improvement in the receiver's predictions was detected. They also show that synthetic text sources are not interchangeable. A second LLM's encyclopedia text worked somewhat less well, while fiction worked much less well. At equal character counts, real text still gave about a third more net gain. Domain matters as well. Relative to each domain's own reference, real- and LLM-text bridges fitted on encyclopedic text kept 73–81% of the content on long reviews but about 30% on code, where the ranking of probe sources by net gain also changed.

**Relation to weight-space reasoning.** The study began from evidence that frontier models can read and edit network weights (Takehi et al., 2026), and in small testbeds an LLM designed complete bridges from weights alone (Appendix F). At the scale of pretrained models, its useful contribution narrowed to supplying in-register text and the distribution-matching recipe it had proposed on the small testbed. Our experiments do not distinguish the contribution of the LLM's world knowledge from those of the text's register and topic coverage. The weight-only map we tested, a least-squares fit between embedding matrices in the spirit of the within-model alignment of LatentMAS (Zou et al., 2026), delivered 2% of the reference. That rules out this particular map, but leaves weight-based alignment in general open.

# 6 Limitations

- **Scale and family.** We test three Pythia sizes (up to 410M) and three cross-family pairs (up to 0.5B), and the results differ widely between pairs. The cross-family runs use two seeds and a restricted layer search, and all models are small.
- **One task, few domains.** We evaluate next-token prediction with bridges fitted on WikiText-like text and tested on WikiText, long movie reviews and Python code. We do not test whether the measured content predicts benefits in downstream collaboration such as question answering or reasoning.
- **Cross-family protocol.** Sender message length is chosen from the test inputs, so the sender often reads less of the prefix than the receiver-state reference (93% of its tokens for GPT-2 into Pythia, 74% into Qwen2.5). We have no position-matched no-context control for receivers with absolute positions, and we read these results as protocol-specific case studies. Two further planned pairs were dropped for compute after the first cross-family run had started (Appendix D). The three reported pairs are therefore not a blind or representative sample, and their intervals exclude pair-selection uncertainty.
- **One receiver design.** The receiver gets a single residual-stream injection at one layer. Multi-layer or key–value injection may raise what latent messages can deliver, and the text-versus-latent comparison is specific to this design.
- **Synthetic corpora.** The main results use one LLM, one prompt and 64 topic areas, and the variant experiment adds one corpus from a second LLM and one fiction corpus. Each is a single generated sample, and the corpora also differ in token count and topic coverage. The second LLM ran through an agentic command-line tool in a read-only sandbox, which restricts tool access without removing it, and we did not pin its model version. We cannot rule out WikiText-like text in either LLM's pretraining.
- **Log-score, not information.** Gains are differences in expected log-score through a fixed receiver. They measure usefulness to that receiver, not the mutual information between message and prefix.
- **Statistics.** Intervals resample test articles, fitting seeds and donor draws, but with two or three fits per condition the seed component is poorly estimated, and the donor pool (other test windows) is

fixed. Per-seed values are reported for the main pair; diagnostics (§4.6) and layer sweeps use one seed. Larger and cross-family pairs use two or three fits, the capacity ablation one.

## Use of AI tools

The experiments were designed, implemented and run with an AI coding assistant (Claude, Anthropic) under the author's direction. The manuscript was also drafted with this assistant under the author's direction. A second model (GPT, OpenAI) then audited the manuscript and code in repeated rounds through the Codex command-line tool with read-only access. It recomputed the reported numbers from the retained per-window arrays. These audits were model-assisted checks and did not constitute independent replication. The models also generated the encyclopedia-style probe text used to fit the synthetic-text bridges (§3). The author takes full responsibility for the content.

## References

Samuel K. Ainsworth, Jonathan Hayase, and Siddhartha Srinivasa. Git re-basin: Merging models modulo permutation symmetries, 2023. URL https://arxiv.org/abs/2209.04836.

Yamini Bansal, Preetum Nakkiran, and Boaz Barak. Revisiting model stitching to compare neural representations, 2021. URL https://arxiv.org/abs/2106.07682.

Stella Biderman, Hailey Schoelkopf, Quentin Anthony, Herbie Bradley, Kyle O'Brien, Eric Hallahan, Mohammad Aflah Khan, Shivanshu Purohit, USVSN Sai Prashanth, Edward Raff, Aviya Skowron, Lintang Sutawika, and Oskar van der Wal. Pythia: A suite for analyzing large language models across training and scaling, 2023. URL https://arxiv.org/abs/2304.01373.

Alan Chen, Jack Merullo, Alessandro Stolfo, and Ellie Pavlick. Transferring linear features across language models with model stitching, 2025. URL https://arxiv.org/abs/2506.06609.

Jiaming Cheng, Subhransu Das, and Rajiv Ramnath. When does latent communication pay? A causal audit of relayed KV caches in multi-agent LLMs, 2026. URL https://arxiv.org/abs/2608.04893.

Zhuoyun Du, Runze Wang, Huiyu Bai, Zouying Cao, Xiaoyong Zhu, Yu Cheng, Bo Zheng, Wei Chen, and Haochao Ying. Enabling agents to communicate entirely in latent space, 2026. URL https://arxiv.org/abs/2511.09149.

Tianyu Fu, Zihan Min, Hanling Zhang, Jichao Yan, Guohao Dai, Wanli Ouyang, and Yu Wang. Cache-to-cache: Direct semantic communication between large language models, 2026. URL https://arxiv.org/abs/2510.03215.

Minyoung Huh, Brian Cheung, Tongzhou Wang, and Phillip Isola. The platonic representation hypothesis, 2024. URL https://arxiv.org/abs/2405.07987.

Rishi Jha, Collin Zhang, Vitaly Shmatikov, and John X. Morris. Harnessing the universal geometry of embeddings, 2026. URL https://arxiv.org/abs/2505.12540.

Karel Lenc and Andrea Vedaldi. Understanding image representations by measuring their equivariance and equivalence, 2015. URL https://arxiv.org/abs/1411.5908.

Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher. Pointer sentinel mixture models, 2016. URL https://arxiv.org/abs/1609.07843.

Luca Moschella, Valentino Maiorca, Marco Fumero, Antonio Norelli, Francesco Locatello, and Emanuele Rodolà. Relative representations enable zero-shot latent space communication, 2023. URL https://arxiv.org/abs/2209.15430.

Rikiya Takehi, Jayden Teoh, Alex Zhang, and Omar Khattab. Sparks of weight-space programming: LLMs can natively read & write neural network weights. Online research article, https://rikiyat.github.io/blog/weight-space-programming/, nov 2026. Experiment logs: https://github.com/RikiyaT/lm_understand-logs.

Hongxu Yin, Pavlo Molchanov, Zhizhong Li, Jose M. Alvarez, Arun Mallya, Derek Hoiem, Niraj K. Jha, and Jan Kautz. Dreaming to distill: Data-free knowledge transfer via deepinversion, 2020. URL https://arxiv.org/abs/1912.08795.

Huixiang Zhang and Mahzabeen Emu. Do latent channels actually communicate? A causal audit of latent multi-agent LLM communication, 2026. URL https://arxiv.org/abs/2607.26773.

Jiaru Zou, Ruizhong Qiu, Gaotang Li, Xiyuan Yang, Katherine Tieu, Pan Lu, Ke Shen, Hanghang Tong, Yejin Choi, Jingrui He, James Zou, Mengdi Wang, and Ling Yang. Latent collaboration in multi-agent systems, 2026. URL https://arxiv.org/abs/2511.20639.

# A Receiver-state reference versus injection depth

We compute $B$'s own layer-$L_B$ prefix states by running $B$ on the prefix alone. We inject these states into $B$'s suffix pass at the same layer. This diagnostic uses 128 test windows and is not used for any selection. The script is `pb_ceiling.py`, and the output is `ceiling.txt`.

| Injection layer $L_B$ | 1 | 2 | 3 | 4 | 6 | 8 |
|---|---|---|---|---|---|---|
| pythia-160m as $B$: accuracy (%) | 34.3 | 33.4 | 32.4 | 31.4 | 26.2 | 25.0 |
| pythia-70m as $B$: accuracy (%) | 28.5 | 27.8 | 25.4 | 20.4 | – | – |

Table 4: References on the same windows: pythia-160m with the full prefix 37.1%, with none 24.0%; pythia-70m 31.9% and 19.8%. Below the injection layer $B$ processes the suffix without the prefix, and the benefit of its own prefix states falls quickly with depth.

# B Layer selection on the validation split

For each $(L_A, L_B)$ we fit a linear bridge on real text with the communication objective (300 steps, seed 0) and score its suffix cross-entropy on 256 non-overlapping WikiText-103 *validation* windows. Selection uses this matched cross-entropy only.

| $L_A$ (pythia-70m) | 1 | 2 | 3 | 4 | 5 | 6 (final, post-LN) |
|---|---|---|---|---|---|---|
| $L_B = 1$ | 4.235 | **4.031** | 4.106 | 4.195 | 4.126 | 4.641 |
| $L_B = 2$ | 4.083 | 4.086 | 4.103 | 4.176 | 4.187 | 4.555 |

Table 5: Validation suffix CE (nats/token). Receiver-state reference on the same windows: 3.908 ($L_B = 1$), 3.969 ($L_B = 2$).

**Selection without corpus text.** We repeated the sweep with bridges fitted on the 24 LLM-written articles used in the main experiments. We scored them on 256 windows from the other 40 LLM-written articles, using no corpus text at any stage.

| $L_A$ | 1 | 2 | 3 | 4 | 5 | 6 |
|---|---|---|---|---|---|---|
| $L_B = 1$ | 3.804 | **3.531** | 3.623 | 3.699 | 3.633 | 4.084 |
| $L_B = 2$ | 3.633 | 3.593 | 3.636 | 3.692 | 3.738 | 4.075 |

Receiver-state reference on these windows: 3.424 ($L_B = 1$), 3.457 ($L_B = 2$). The sweep selects the same pair as the WikiText validation sweep.

# C Truncated messages: absolute cross-entropy

Suffix CE (nats/token) when only the last $K$ prefix positions are sent; fitted bridges are the mean of three seeds. $B$ with no context: 4.924.

| $K$ | 1 | 2 | 4 | 8 | 16 | 32 | 64 | 128 |
|---|---|---|---|---|---|---|---|---|
| Text (last $K$ tokens) | 4.822 | 4.747 | 4.627 | 4.447 | 4.221 | 3.978 | 3.746 | 3.572 |
| Receiver-state reference | 5.127 | 5.151 | 5.115 | 4.970 | 4.785 | 4.527 | 4.266 | 3.836 |
| Real-text bridge (comm) | 5.233 | 5.294 | 5.286 | 5.200 | 5.004 | 4.717 | 4.434 | 3.827 |
| LLM-text bridge (comm) | 5.205 | 5.320 | 5.390 | 5.369 | 5.258 | 5.080 | 4.871 | 4.137 |

Table 6: Short latent messages are worse than no context in absolute terms, including $B$'s own states (reference, $K \leq 8$). A message of $K < P$ positions places a mid-document state at sequence position 0, where Pythia normally sees the start of a text, and the bridges were fitted only with 128-position messages. We have not isolated which of these causes the drop. The content contrasts in Figure 1c compare matched and other-article messages of the same length and may also be affected.

# D Generalisation experiments: populations and protocol differences

**Populations.** Pythia and cross-family rows of Table 2 with pythia-160m as receiver use the main pair's 493 windows from 54 articles; the pythia-410m receiver uses the same windows. GPT-2→Qwen2.5 and OPT→GPT-2 define windows in their receivers' tokenisations; after the same restrictions each analyses 484 windows from 53 articles. Domain windows are listed in `dom_code_windows.json` and `dom_reviews_windows.json` (directory `out_pb`), a retrospective reconstruction from the same inputs and selection rules whose document identities match the saved runs (the original runs did not store offsets). *Code*: top-level Python standard-library files with at least two non-overlapping 161-token windows; up to four windows per file were drawn and 512 sampled from them (157 files); the decomposition uses the 509 windows from 154 files that have a same-file donor. *Reviews*: IMDB test reviews with at least 322 Pythia tokens (7,440 of 25,000, i.e. long reviews only); 512 windows from 501 reviews. Only 22 windows have a same-review donor, so for reviews we estimate generic and content parts on all 512 windows and do not split content into article-level and window-specific parts. Donors in both domains come from other documents, with five random draws per window. Intervals resample documents, fitting seeds and donor draws, and are conditional on the selected documents and the fixed donor pool.

**Irrelevant messages beyond the main pair.** Generic parts (no context minus other-document donor, nats): pythia-160m→pythia-410m, real-text bridge $-0.07$ [$-0.12$, $-0.02$], text $-0.18$ [$-0.23$, $-0.13$]; reviews, real-text bridge $-0.02$ [$-0.04$, $-0.00$], text $+0.04$; code, real-text bridge $-0.35$ [$-0.38$, $-0.32$], text $-0.26$ [$-0.29$, $-0.24$].

**Cross-family protocol.** Five cross-family pairs were planned. We reduced the suite about half an hour after the first run, GPT-2 into Pythia, had started. Some results may already have been printed. To fit the compute budget, we dropped Pythia-160M into SmolLM2-135M and Pythia-160M into GPT-2. We also dropped self-sample probes for GPT-2 into Qwen2.5 and OPT into GPT-2 (`out_pb/x_skip.txt`, `x_lean.txt`). None of the dropped fits was run, but the choice of reported pairs was not blind. Windows and targets are defined in the receiver's tokenisation ($P_B = 128$ prefix and $S = 32$ suffix tokens). The sender receives the decoded prefix text, re-tokenised with its own tokenizer and truncated to its last $P_A$ tokens so that every message has the same length. $P_A$ is the shortest re-tokenised prefix among the test windows ($P_A = 118$ for GPT-2 into Pythia; prefix lengths range from 118 to 151, median 127). This choice uses test inputs but no targets; it truncates 511 of 512 prefixes and keeps on average 93% of the sender's prefix tokens, while the receiver-state reference uses all 128 receiver-token states. For GPT-2 into Qwen2.5, $P_A = 90$ (prefix lengths 90 to 146, median 123; prefix length correlates at $-0.85$ with the share of digits, which Qwen tokenises one per token), so the sender keeps on average only 74% of its prefix tokens. Probe and validation windows whose sender prefix is shorter than $P_A$ are dropped. For the real-text and LLM-written probes, 4029 to 4094 of 4096 windows are kept per pair and seed, and 253 to 256 of 256 validation windows (listed in `out_pb/general_audit.json`). The GPT-2→Pythia self-sample probes (two seeds) were generated in memory and not saved, so their filtering counts are not available. Suffix states are computed with position ids that start at the message length. For receivers with absolute position embeddings (GPT-2 as receiver), the no-context control starts at position 0, so generic and net parts may include a position-offset effect; content contrasts compare matched and donor messages at identical positions. The weight-only map is fitted on tokens whose strings occur in both vocabularies. The cross-family runner forms self-sample probe text by concatenating decoded samples and drawing windows from it, unlike the Pythia runner, which uses the generated sequences directly.

**Text variants.** The variant corpora were concatenated in file order and truncated to 371,254 characters each, which keeps 22 complete encyclopedia articles from the first LLM (85k Pythia tokens), 20 from the second (72k tokens) and all 24 fiction texts (96k tokens). Four fiction generations were interrupted when the machine slept and were regenerated with the same prompt; only the final files are archived.

# E Reproducibility

**Data.** WikiText-103 raw (Merity et al., 2016): the first 30,000 training lines (8.8M characters) form the real-text pool; the first 6,000 test lines give 512 non-overlapping 161-token test windows (seed 123); the first 6,000 validation lines give 256 windows for layer selection. **LLM-written text.** Each article was requested with the prompt "Write about 2,500 words of original encyclopedia articles in the plain, factual style of Wikipedia, with section headings written like '= = History = ='. Topic area: *X* (invent plausible specifics freely; do not copy any real article). Output only the article text." using Claude Opus 5.5 at low effort with all tools disabled, an empty working directory and no session persistence. Of 20,604 non-overlapping 50-character windows of the 1.03M-character collection, 2 occur verbatim in the test text and 1 in the validation text; all three are stock phrases (e.g. "…by the Recording Industry Association of Amer[ica]"). **Self-samples.** 512 sequences, 128 each from two seeds of $A$ and of $B$, unconditional from the end-of-text token at temperature 1. **Compute.** One Apple M5 Pro (Metal backend, float32). A 600-step fit of the 70M→160M bridge together with its evaluation took 4–6 minutes when run alone. Timing ledger (process time, several processes sharing one GPU): the timed fits in the logs of the main, generalisation and diagnostic runs (`out_pb/*.log`) total 22.8 hours, and the earlier Pythia seed runs (`out_pythia/seeds.log`) 1.1 hours; layer sweeps and donor evaluations were not timed, and the timed parts of aborted runs are excluded. The main and generalisation suite ran from 3 November 12:30 (first log) to 4 November 04:58 GMT (last result file), 16.5 hours of wall-clock time; the ear-

lier Pythia runs and later audit scripts fall outside this interval. **Code.** Scripts, logs, per-window results, fitted bridges, probe texts, pinned environments (`requirements-lock.txt`), the Hugging Face revisions of every model and dataset (enforced in the loaders) and a manifest of full SHA-256 file hashes are kept together, with the run scripts actually used (`out_pb/queue*.sh`). The archive supports numerical reanalysis of every reported result; reconstruction of the experiments is documented for the main `pb*` suite, whereas the small-testbed harness of Appendix F needs a sandbox set-up that is only partly documented. Model and dataset revisions are pinned in all retained loaders. The archive is released at https://github.com/dongzhaohe321418-lab/content-or-context. Not kept: transcripts of the second LLM's generations, failed fiction generations that were overwritten when regenerated, and all self-sample probe text (main, larger-Pythia and cross-family runs), which was generated in memory; the resulting bridges and evaluation arrays are kept. Probe windows are drawn with a sampler that never selects the last legal start position of the probe text; we kept it unchanged so that saved windows reproduce.

# F Small-scale studies: an LLM designs the whole bridge from weights

Before the Pythia study we ran two controlled testbeds in which the LLM (Claude Opus 5.5) designs the *entire* bridge, not only the probe text, from the two models' weights. These results motivated the KL-matching recipe used in the main text, which the LLM itself proposed in the second testbed.

**Conditions.** (i) *Tool-free*: one turn, the weights printed as text, no code execution. (ii) *Weight algebra only*: the model submits one Python program per turn, executed in a macOS `sandbox-exec` jail (no network, no read access to the workspace), and may not construct any input or run either network. (iii) *Code and self-made probes*: as (ii), but it may synthesise inputs. In no condition does it see any training or test data. Every turn was logged and audited.

**Digit classifiers.** Three pairs of two-hidden-layer MLPs trained on sklearn's $8 \times 8$ digits from different seeds: same width (16-16 and 16-16, ReLU); different width (16-16 and 24-24, ReLU); different width and activation (16-16 ReLU and 24-16 tanh). The bridge maps $A$'s first hidden layer to $B$'s first pre-activation; the score is $B$'s accuracy on 594 held-out images (95% Wilson half-widths are about 1.5–2 points near 95% accuracy and about 4 points near 50%).

| Method | same | width | hetero |
|---|---|---|---|
| $A$ alone / $B$ alone | 96.0 / 96.0 | 96.5 / 97.5 | 96.0 / 96.5 |
| Random bridge | 10.0 | 10.5 | 8.5 |
| Unit matching (Re-Basin style, data-free) | 32.3 | 33.8 | 40.1 |
| Weight-space least squares (data-free algorithm) | 84.3 | 89.9 | 90.1 |
| Best noise probe (selected on test; favours the baseline) | 88.0 | 93.6 | 89.1 |
| LLM, tool-free (3 runs: mean [min–max]) | 43.4 [20–82] | 63.2 [47–82] | 39.4 [10–73] |
| LLM, weight algebra only (1 run) | 83.3 | 95.3 | 87.7 |
| **LLM, code + self-made probes (2 runs)** | **94.9** | **94.9** | **93.9** |
| Fitted on real training images (upper reference) | 95.3 | 96.8 | 94.1 |

Table 7: Digit classifiers. With code, the LLM synthesised digits from stroke templates and fitted the bridge on them; pool sizes and augmentation differed between runs (per-run transcripts are archived). It exceeds the best noise probe by 1.3–6.9 points; we did not save per-image predictions, so we report no paired significance test, and the smallest difference (different-width pair, +1.3) should be treated as unresolved. Without tools it derives the right principle, $W_1^A M \approx W_1^B$, but cannot invert matrices by hand, hence the 10–82% spread.

**Character-level GPTs.** Two 4-layer character-level transformers trained from different seeds on Python standard-library source, widths 64 and 96; the bridge connects their residual streams after layer 2 and the score is $B$'s next-character accuracy. $A$ alone scores 62.8 and $B$ alone 63.6. A bridge fitted on real code by minimising $B$'s next-character cross-entropy against the true characters reaches 55.58 (three seeds, s.d. 0.11). In condition (iii), the LLM started from the weights alone and implemented the forward pass from a textual specification. It derived a backward pass and checked it against numerical gradients. It then fitted the bridge on probe text it wrote itself and text sampled from $A$ and $B$. The two runs used different objectives: run 0 matched $B$'s own predictive distribution, run 1 matched an equal mixture of $A$'s and $B$'s distributions. They reached 55.0 and 55.7. Fitting the hard-target objective on text sampled from $A$ and $B$, with no LLM involved, reaches 54.95 (s.d. 0.06). These recipes are not algorithm-matched, so the comparison shows that self-sampling suffices on this testbed, not that the LLM's recipe is uniquely good. Python probe text written by the LLM without tools (52k characters from four calls) reaches 49.80 (s.d. 0.09), against 26.88 for the same characters shuffled. Two of the four calls returned a refusal and a meta-response instead of code, and we did not filter them, so 4.6% of that probe file is not code. Of its non-overlapping 50-character windows, 1.1% and 0.9% (the two code-bearing calls) occur verbatim in the standard-library corpus from which the models' training split is taken (0.9% and 0.9% in the training split itself).

**Two observations carried into the main study.** First, in the hand-over setting the bridged system scores below $A$ alone in accuracy on both language testbeds (55.7 vs 62.8 here; 21.1 vs 25.7 on Pythia). Section 4.6 shows that this gap depends on training length, capacity and metric, so it should not be read as information loss. Second, self-sampling is sufficient for tiny models trained on a narrow corpus, which raises the question of whether it remains sufficient for pretrained models with broad distributions. Section 4 answers both.

**A note on refusals.** One of the four archived calls asking the LLM to write probe programs for "calibrating a connection between two models" declined, reading the request as a covert-channel or steganography exercise; another returned a meta-response without code. The encyclopedia-article calls used a neutral description of the task (Appendix E). We archived 64 outputs. Two numbered files from the first batches are missing, for reasons we did not record.